Security Log Analysis: Technical Documentation

1. Project Overview

Security Log Analysis is a web application that lets an administrator upload predefined security log files, parses them into a database, detects suspicious activity with rule-based analysis, and shows the results on a dashboard with filters, charts and downloadable reports.

Objectives

- Automate the manual reading of security logs.
- Detect common threats (brute force, unauthorized access, abnormal IP activity) using transparent, explainable rules.
- Present findings visually so an analyst can triage quickly.

Scope (Version 1)

- One role: Administrator.
- One or two well-defined log formats (Apache access log and a fixed-header CSV).
- Rule-based detection only (no machine learning).
- Synchronous processing of small files (limitation: very large files are not supported).

Technology stack

Layer	Technology	Why it is used
Front end	HTML5, CSS3, Bootstrap, JavaScript, Chart.js	Simple, no build step, responsive cards/tables, charting
Back end	Node.js, Express.js	JavaScript on both sides, easy REST APIs
Data base	MySQL	Relational data with joins, indexes, filtering
Upload	Multer	Handles multipart file uploads
Security	bcrypt, JWT, helmet, express-rate-limit, cors, dotenv	Password hashing, tokens, headers, throttling, config
Parsing	Built-in `readline` (text logs), `csv-parser` (CSV)	Stream large files line by line

---

2. System Architecture

```
Browser (HTML + Bootstrap + JS + Chart.js)
        │  REST API / JSON  (Authorization: Bearer <JWT>)
        ▼
Node.js + Express.js
  routes → middleware → controllers → services → database
        │  parameterized SQL (mysql2)
        ▼
MySQL (users, log_files, log_entries, security_incidents)
```

Layer responsibilities

- **Routes**: map URLs to controller functions (`POST /api/logs/upload`).
- **Middleware**: code that runs before a controller (auth check, error handling, upload restrictions).
- **Controllers**: read the request, call services, send the response. No business logic here.
- **Services**: the real logic (`parserService.js`, `detectionService.js`). Kept separate so they can be tested without HTTP.

- **Database layer**: a `mysql2` connection pool in `config/database.js`.

End-to-end data flow

```
Admin login → Upload file → Validate → Parse lines → Store in MySQL
→ Run detection rules → Store incidents → Dashboard → Report
```

---

3. Folder Structure and Purpose

```
security-log-analysis/
├── frontend/   login.html, dashboard.html, upload.html, incidents.html, reports.html
│   ├── css/style.css
│   └── js/     login.js, dashboard.js, upload.js, incidents.js
├── backend/
│   ├── server.js            entry point
│   ├── .env                 secrets (never commit)
│   ├── config/database.js   MySQL pool
│   ├── routes/              authRoutes, logRoutes, incidentRoutes, dashboardRoutes
│   ├── controllers/         request handlers
│   ├── services/            parserService.js, detectionService.js
│   ├── middleware/          authMiddleware.js, errorMiddleware.js
│   └── uploads/             stored log files (outside public folders)
└── database/schema.sql
```

---

4. Database Design

4.1 Tables and relationships

```
users 1 ──< log_files 1 ──< log_entries 1 ──< security_incidents
                └──────────────────────────────────^ (file_id also stored on incidents)
```

Table	Purpose	Key columns
`users`	Administrator accounts	`username` (unique), `password_hash`, `role`
`log_files`	One row per upload	`original_name`, `stored_name`, `file_type`, `file_size`, `uploaded_by`, `status`
`log_entries`	One row per parsed log line	`file_id`, `event_time`, `source_ip`, `username`, `method`, `endpoint`, `status_code`, `user_agent`, `event_type`, `raw_message`
`security_incidents`	One row per detection	`log_entry_id`, `file_id`, `incident_type`, `severity`, `source_ip`, `description`, `evidence`, `status`

4.2 Design reasoning

- **Separate `original_name` and `stored_name`**: the user's filename is never trusted; the server generates its own name.
- **`raw_message` is kept**: the original line is the evidence, so analysts can verify a detection.
- **`log_entry_id` is nullable on incidents**: aggregate incidents (e.g., 8 failed logins) relate to many entries, not one.
- **`BIGINT` ids on entries and incidents**: log tables grow fast.
- **`VARCHAR(45)` for IPs**: long enough for IPv6.
- **Foreign keys** keep data consistent (no entry without a file).

4.3 Indexes and why

Index	Speeds up
`log_entries(event_time)`	Timeline chart, date filters, time-window detection
`log_entries(source_ip)`	Per-IP counts, "top IPs" chart
`security_incidents(severity)`	Severity filter and pie chart
`security_incidents(detected_at)`	Date range filter, recent incidents

4.4 Recommended additions

- `incident_count INT DEFAULT 1` and `signature VARCHAR(255) UNIQUE` on `security_incidents` (duplicate prevention, section 8.4).
- `valid_lines` and `invalid_lines` on `log_files` (to show parse results).
- `audit_log` table (user_id, action, timestamp) for the audit module.

---

5. Backend: How It Works

5.1 `server.js` (startup)

1. Load `.env` with `dotenv`.
2. Create the Express app and apply `helmet()`, `cors()`, `morgan()`, and `express.json()`.
3. Mount routes under `/api/auth`, `/api/logs`, `/api/incidents`, `/api/dashboard`, `/api/reports`.
4. Serve the `frontend/` folder as static files.
5. Register `errorMiddleware` **last** so it catches errors from everything above.
6. Test the DB connection, then `app.listen(PORT)`.

5.2 Authentication logic

```
POST /api/auth/login {username, password}
  → SELECT user WHERE username = ?          (parameterized)
  → bcrypt.compare(password, password_hash)
  → jwt.sign({id, role}, JWT_SECRET, {expiresIn: '2h'})
  → return {token}
```

- **Why bcrypt**: it is slow and salted on purpose, so stolen hashes are hard to crack.
- **Why JWT**: the server stays stateless; every request carries proof of identity.
- **`authMiddleware.js`**: reads `Authorization: Bearer <token>`, runs `jwt.verify`, attaches `req.user`, and returns **401** on a missing or invalid token. It wraps every protected route.
- **Login rate limit**: `express-rate-limit` (for example 5 attempts per 15 minutes per IP) blocks brute-forcing of the login itself.
- Return the same message ("Invalid credentials") for a wrong username or password so attackers cannot discover valid usernames.

5.3 Secure upload (Multer)

Configuration logic:

- `diskStorage` with a generated name (`Date.now() + random + .ext`), destination `uploads/` outside any public folder.
- `fileFilter`: allow only `.log`, `.txt`, `.csv`.
- `limits.fileSize = MAX_FILE_SIZE` (10 MB from `.env`).
- After upload: reject 0-byte files, sanity-check the first line against the expected format, and delete the file if invalid.

Controller flow:

```
insert log_files row (status='uploaded')
→ set status='processing'
→ parserService.parse(file)  → entries[], invalidCount
→ batch insert entries
→ detectionService.run(fileId)
→ set status='completed' | 'completed_with_errors' | 'failed'
→ respond {message, fileId, status, validLines, invalidLines}
```

5.4 Parser service (`parserService.js`)

Goal: turn different raw formats into one common object so detection does not care about the source.

Apache line

```
192.168.1.25 - - [01/Sep/2026:09:15:10 +0530] "GET /admin HTTP/1.1" 401 532
```

Parsed with a regular expression with capture groups:

```
^(\S+) \S+ \S+ \[(.+?)\] "(\S+) (\S+) [^"]*" (\d{3}) (\d+|-)
```

Group	Field
1	`sourceIp`
2	`eventTime` (convert `01/Sep/2026:09:15:10 +0530` to `2026-09-01 09:15:10`)
3	`method`
4	`endpoint`
5	`statusCode`

CSV (required header `timestamp,ip,username,event_type,status,endpoint,message`): `csv-parser` streams rows; each is validated (non-empty IP, valid timestamp, numeric status) and mapped to the same object.

Algorithm

```
for each line (streamed):
    if it matches the format and passes validation:
        push to batch; when batch reaches 500 → bulk INSERT, clear batch
    else:
        invalidCount++ (optionally record the line number)
flush remaining batch
```

One bad line never stops the run. Batching reduces thousands of INSERTs to a few.

5.5 Threat detection service (`detectionService.js`)

All thresholds live in **one config object** so they are never hard-coded across files:

```
const RULES = {
  bruteForce:   { attempts: 5, windowMin: 10, highAt: 10 },
  abnormalIp:   { requests: 100, windowMin: 5 },
  errorSpike:   { errors: 30, windowMin: 5 },
  sensitivePaths: ['/admin','/login','/config','/.env','/wp-admin'],
  injection:    [/'\s*or\s*1=1/i, /\.\.\//, /<script/i, /union\s+select/i]
};
```

Each rule is a separate function that reads `log_entries` for the file, so rules can be added or tested independently.

#	Rule	Logic	Severity
1	Repeated failed logins	Group by IP; count entries with failure status (401 on login path, or `event_type='LOGIN_FAILED'`) in a sliding 10-minute window; ≥ 5 triggers	5–9 Medium, ≥ 10 High
2	Unauthorized access	Status 401 or 403	Low; raise to Medium if the same IP repeats on protected paths
3	Abnormal IP activity	> 100 requests from one IP in 5 minutes	Medium/High
4	Suspicious endpoint	Path matches the sensitive list	Low ("suspicious", not "attack")
5	Error-rate anomaly	Many 4xx/5xx from one IP in a window	Medium
6	Suspicious input	Endpoint or message matches an injection pattern	Medium (pattern-based, not proof)

Time-window (sliding window) idea for rules 1, 3, 5:

```
sort an IP's matching events by time
for each event as the window start:
    count events within [start, start + window]
    if count ≥ threshold → create incident, skip past this window
```

This can be done in JavaScript after one SQL query, or with SQL `GROUP BY source_ip, FLOOR(UNIX_TIMESTAMP(event_time)/600)` for fixed windows (simpler; fine for V1).

5.6 Preventing duplicate incidents

Build a signature: `incident_type + source_ip + window_start`. Store it in a `UNIQUE` column and use `INSERT ... ON DUPLICATE KEY UPDATE incident_count = incident_count + 1`. One IP hammering the server for an hour then produces a few grouped incidents, not thousands.

5.7 Evidence storage

Every incident stores *why* it fired, for example:

> *IP 192.168.1.25 generated 8 failed login attempts between 09:10 and 09:20. Threshold: 5 attempts.*

Also store related entry IDs and the raw lines. This makes each detection explainable and defensible during evaluation.

5.8 Dashboard and report queries

All aggregation is done in SQL, never in the browser:

- **Summary**: `COUNT(*)` on files, entries, failed logins (status 401/failed events), incidents; top IPs via `GROUP BY source_ip ORDER BY COUNT(*) DESC LIMIT 5`.
- **Timeline**: `GROUP BY DATE_FORMAT(event_time,'%Y-%m-%d %H:00')`.
- **Severity**: `GROUP BY severity` → `{labels:[...], values:[...]}`.
- **CSV report**: same query as the incident list with the filters applied, streamed with headers `Content-Type: text/csv` and `Content-Disposition: attachment`.

5.9 Error handling

`errorMiddleware.js` catches all thrown errors, logs details on the server only, and returns a safe response with no stack trace:

```
{ "success": false, "message": "Invalid log file format", "errorCode": "INVALID_FORMAT" }
```

Status codes: 200 OK, 201 created, 400 bad input, 401 unauthenticated, 403 forbidden, 404 not found, 500 server error.

---

6. REST API Reference

Method	Endpoint	Auth	Purpose
POST	`/api/auth/login`	No	Returns JWT
GET	`/api/auth/me`	Yes	Current user
POST	`/api/logs/upload`	Yes	Upload and analyze file (`logFile` field)
GET	`/api/logs`	Yes	Uploaded file history
GET	`/api/logs/:id`	Yes	One file's details and counts
GET	`/api/log-entries`	Yes	Search entries (`?ip=`, `?page=`)
GET	`/api/incidents`	Yes	List with filters `severity, status, type, ip, from, to, page`
GET	`/api/incidents/:id`	Yes	Incident with evidence and raw lines
PATCH	`/api/incidents/:id/status`	Yes	open, investigating, resolved
GET	`/api/dashboard/summary`	Yes	Card values
GET	`/api/dashboard/timeline`	Yes	Events over time
GET	`/api/dashboard/severity`	Yes	Severity counts

Method	Endpoint	Auth	Purpose
GET	`/api/reports/incidents.csv`	Yes	CSV download

Filtering uses query parameters. Build the SQL `WHERE` clause dynamically but always with `?` placeholders and a values array, never string concatenation. **Pagination** uses `LIMIT ? OFFSET ?` and returns `{data, page, total}`.

---

7. Frontend: How It Works

7.1 General pattern (all pages)

1. On load, read the token from `sessionStorage`; if missing, redirect to `login.html`.
2. Call the API with `fetch` and the `Authorization` header.
3. If the response is **401**, clear the token and redirect to login.
4. Render results into the DOM; show loading and empty states.

A shared helper avoids repetition:

```
async function api(path, options = {}) {
  const token = sessionStorage.getItem('token');
  const res = await fetch('/api' + path, {
    ...options,
    headers: { ...(options.headers || {}), Authorization: `Bearer ${token}` }
  });
  if (res.status === 401) { sessionStorage.clear(); location.href = 'login.html'; return; }
  return res.json();
}
```

7.2 Pages

Page	Elements	Logic
Login	Username, password, button, spinner, error box	Validate non-empty → disable button → POST login → store token → redirect; show "Invalid credentials" on failure
Dashboard	4 Bootstrap cards, 3 Chart.js charts, top-IP list, recent incidents	Calls summary, timeline, severity in parallel (`Promise.all`); passes `labels` and `values` to Chart.js
Upload	File input, format info, progress bar, history table	Validate extension, size and non-empty; send `FormData`; show file ID, status, valid/invalid counts; refresh history
Incidents	Filter form, table, pagination	Read filters → build query string → fetch → clear table → render rows or "No results"
Log details	Entry fields and raw line	Shows the original entry plus any related incident
Reports	Date range, severity, type, download button	Builds the report URL from filters and downloads the CSV

7.3 Key code patterns

Upload (do *not* set `Content-Type` manually; the browser adds the multipart boundary):

```
const formData = new FormData();
formData.append('logFile', selectedFile);
fetch('/api/logs/upload', { method:'POST', headers:{ Authorization:`Bearer ${token}` }, body: formData }
      );
```

Chart.js:

```
new Chart(ctx, { type:'doughnut',
  data:{ labels: data.labels, datasets:[{ data: data.values }] } });
```

On filter change, destroy or update the chart with new data instead of reloading the page.

7.4 Frontend validation

Empty file, wrong extension, oversize file, empty login fields, duplicate submit (disable the button during requests), and clear messages. This is only for usability; the **backend repeats every check**.

7.5 XSS protection

Log content is untrusted and may contain `<script>`. Render it with `textContent` (never `innerHTML`) or escape it first.

---

8. Security Design

Threat	Control
Stolen password database	bcrypt hashes
SQL injection	Parameterized queries everywhere
Unauthorized API use	JWT middleware on all admin routes
Login brute force	Rate limiting
Malicious uploads	Extension and size limits, server-generated names, content check, storage outside public folders, path-traversal prevention
XSS from logs	Escape or `textContent`
Info leakage	Central error handler, no stack traces
Header attacks	`helmet`
Secret exposure	`.env`, added to `.gitignore`
Accountability	Audit log of uploads, logins, status changes

---

9. Testing Documentation

Sample files (document expected results for each):

File	Expected outcome
`normal_access.log`	N entries, **0 incidents** (proves no false alarms)
`failed_login.log`	5+ failures from one IP → Brute Force incident
`unauthorized_access.log`	Repeated 401/403 → Unauthorized Access
`abnormal_ip_activity.log`	> 100 requests in 5 min → Abnormal Activity
`mixed_security_events.log`	Several incident types
`invalid_format.log`	Rejected or counted as invalid lines

Test layers: frontend (empty fields, bad extension, oversize, filters, charts, mobile view), backend (DB connection, missing/invalid token, malformed lines, invalid query parameters), detection (one test file per rule, plus a boundary test at exactly 4 vs. 5 failures).

---

10. Limitations and Future Scope

Limitations: one or two log formats, synchronous processing, rule-based only (fixed thresholds), single admin role.

Future scope: multiple roles, PDF reports, asynchronous processing queue, IP geolocation, email alerts, additional log formats, machine-learning anomaly detection.

---

11. Build Order and Timeline

1. Requirements, use cases, architecture, DB design

2. MySQL schema and Node.js setup
3. Authentication and frontend layout
4. Upload and validation
5. Parser (first format)
6. Storage and file status
7. Detection rules
8. Incident APIs and table
9. Dashboard APIs and charts
10. Filters, reports, responsive design
11. Security testing and optimization
12. Documentation, screenshots, presentation

MVP checklist: login, upload one format, parse and store, failed-login and unauthorized detection, incident table, dashboard cards, two charts, CSV export.

12. Demonstration Flow

Login → empty dashboard → upload sample → show validation and processing → parsed count → incidents → open one and explain the evidence → filter → charts → download report → upload a normal file and show **no false alerts**.

Part 2: Deep Dive (Code-Level Explanation)

13. Backend Code Walkthrough

13.1 `config/database.js`: the connection pool

```
const mysql = require('mysql2/promise');
module.exports = mysql.createPool({
  host: process.env.DB_HOST, user: process.env.DB_USER,
  password: process.env.DB_PASSWORD, database: process.env.DB_NAME,
  waitForConnections: true, connectionLimit: 10
});
```

Why a pool: opening a new connection per request is slow. A pool keeps up to 10 open and lends them out. `mysql2/promise` lets us use `async/await` instead of callbacks. Usage everywhere: `const [rows] = await db.query('SELECT ... WHERE id = ?', [id]);` The `?` placeholders are what make queries injection-safe: the driver sends values separately from the SQL text.

13.2 `server.js`: order matters

```
require('dotenv').config();
const express = require('express'), helmet = require('helmet'), cors = require('cors'), morgan = require
      ('morgan');
const app = express();
app.use(helmet()); app.use(cors()); app.use(morgan('dev')); app.use(express.json());
app.use(express.static('../frontend'));
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/logs', require('./routes/logRoutes'));
app.use('/api/incidents', require('./routes/incidentRoutes'));
app.use('/api/dashboard', require('./routes/dashboardRoutes'));
app.use(require('./middleware/errorMiddleware'));   // MUST be last
app.listen(process.env.PORT, () => console.log('Server running'));
```

Express runs middleware **top to bottom**. Security middleware must come before routes; the error handler must come after them or it never sees their errors.

13.3 Login controller

```
exports.login = async (req, res, next) => {
  try {
    const { username, password } = req.body;
    if (!username || !password)
      return res.status(400).json({ success:false, message:'Username and password required', errorCode:'
      VALIDATION' });
    const [rows] = await db.query('SELECT * FROM users WHERE username = ?', [username]);
    const user = rows[0];
    const ok = user && await bcrypt.compare(password, user.password_hash);
    if (!ok) return res.status(401).json({ success:false, message:'Invalid credentials', errorCode:'AUTH
      _FAILED' });
    const token = jwt.sign({ id:user.id, role:user.role }, process.env.JWT_SECRET, { expiresIn:'2h' });
    res.json({ success:true, data:{ token }, message:'Login successful' });
  } catch (e) { next(e); }
};
```

Logic notes: `next(e)` hands any error to the error middleware. One shared message for "no such user" and "wrong password" avoids user enumeration. To create the first admin, run a one-time seed script that does `bcrypt.hash('yourPassword', 10)` and inserts the row (never store plain text).

13.3.1 `authMiddleware.js`

```
module.exports = (req, res, next) => {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ success:false, message:'Token missing', errorCode:'NO_TOKEN'
       });
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ success:false, message:'Invalid or expired token', errorCode:'BAD_TOKEN
      ' }); }
};
```

Usage: `router.get('/', auth, controller.list)`. A JWT is signed, not encrypted: anyone can read its payload, so never put passwords in it. Tampering breaks the signature and `verify` throws.

13.4 Upload route configuration

```
const multer = require('multer'), path = require('path'), crypto = require('crypto');
const storage = multer.diskStorage({
  destination: path.join(__dirname, '../uploads'),
  filename: (req, file, cb) =>
    cb(null, Date.now() + '-' + crypto.randomBytes(8).toString('hex') + path.extname(file.originalname).
      toLowerCase())
});
const upload = multer({
  storage, limits: { fileSize: Number(process.env.MAX_FILE_SIZE) },
  fileFilter: (req, file, cb) =>
    ['.log','.txt','.csv'].includes(path.extname(file.originalname).toLowerCase())
      ? cb(null, true) : cb(new Error('UNSUPPORTED_TYPE'))
});
router.post('/upload', auth, upload.single('logFile'), controller.upload);
```

`upload.single('logFile')` must match the `FormData` field name exactly. The random name blocks path traversal (`../../x`) and overwrites. Map Multer's `LIMIT_FILE_SIZE` and `UNSUPPORTED_TYPE` errors to 400 in the error middleware.

13.5 Parser service in full

```
const fs = require('fs'), readline = require('readline');
const APACHE = /^(\S+) \S+ \S+ \[(.+?)\] "(\S+) (\S+) [^"]*" (\d{3}) (\d+|-)/;
const MONTHS = {Jan:'01',Feb:'02',Mar:'03',Apr:'04',May:'05',Jun:'06',Jul:'07',Aug:'08',Sep:'09',Oct:'10
      ',Nov:'11',Dec:'12'};

function toSqlTime(s) {                          // "01/Sep/2026:09:15:10 +0530"
  const m = s.match(/(\d{2})\/(\w{3})\/(\d{4}):(\d{2}:\d{2}:\d{2})/);
  return m && MONTHS[m[2]] ? `${m[3]}-${MONTHS[m[2]]}-${m[1]} ${m[4]}` : null;
}
function parseApacheLine(line) {
  const m = line.match(APACHE); if (!m) return null;
  const t = toSqlTime(m[2]); if (!t) return null;
  return { eventTime:t, sourceIp:m[1], method:m[3], endpoint:m[4].slice(0,500),
           statusCode:+m[5], username:null, userAgent:null, eventType:null, rawMessage:line };
}
async function parseFile(filePath, fileId, db) {
  let valid = 0, invalid = 0, batch = [];
  const rl = readline.createInterface({ input: fs.createReadStream(filePath), crlfDelay: Infinity });
  const flush = async () => {
    if (!batch.length) return;
    await db.query('INSERT INTO log_entries (file_id,event_time,source_ip,username,method,endpoint,statu
      s_code,user_agent,event_type,raw_message) VALUES ?',
      [batch.map(e => [fileId,e.eventTime,e.sourceIp,e.username,e.method,e.endpoint,e.statusCode,e.userA
      gent,e.eventType,e.rawMessage])]);
    batch = [];
  };
  for await (const line of rl) {
    if (!line.trim()) continue;
    const e = parseApacheLine(line);
    if (e) { batch.push(e); valid++; if (batch.length >= 500) await flush(); } else invalid++;
  }
  await flush();
  return { valid, invalid };
}
```

How it works: a streamed read keeps memory flat for big files. The regex is the "grammar" of a valid line, and a non-match counts as invalid instead of crashing. The `VALUES ?` with an array of arrays is mysql2's **bulk insert**. Status: `failed` if valid = 0, `completed_with_errors` if invalid > 0, else `completed`.

13.6 Detection service in full

```
function slidingWindow(events, windowMs, threshold) {   // events sorted by time
  const groups = []; let i = 0;
  while (i < events.length) {
    let j = i;
    while (j < events.length && events[j].t - events[i].t <= windowMs) j++;
    if (j - i >= threshold) { groups.push(events.slice(i, j)); i = j; } else i++;
  }
```

```
  return groups;
}

async function bruteForce(fileId, db) {
  const [rows] = await db.query(
    `SELECT id, source_ip, event_time FROM log_entries
     WHERE file_id = ? AND status_code = 401 AND endpoint LIKE '%login%' ORDER BY source_ip, event_time`
      , [fileId]);
  const byIp = {};
  rows.forEach(r => (byIp[r.source_ip] ||= []).push({ id:r.id, t:new Date(r.event_time).getTime(), time:
      r.event_time }));
  for (const [ip, evs] of Object.entries(byIp)) {
    for (const g of slidingWindow(evs, RULES.bruteForce.windowMin * 60000, RULES.bruteForce.attempts)) {
      const severity = g.length >= RULES.bruteForce.highAt ? 'high' : 'medium';
      const start = g[0].time.toISOString().slice(0,16);
      const signature = `BRUTE_FORCE|${ip}|${start}`;
      const evidence = `IP ${ip} generated ${g.length} failed logins in ${RULES.bruteForce.windowMin} mi
      n. Threshold: ${RULES.bruteForce.attempts}. Entry IDs: ${g.map(x=>x.id).join(',')}`;
      await db.query(
        `INSERT INTO security_incidents (log_entry_id,file_id,incident_type,severity,source_ip,descripti
      on,evidence,signature)
         VALUES (?,?,?,?,?,?,?,?) ON DUPLICATE KEY UPDATE incident_count = incident_count + 1`,
        [g[0].id, fileId, 'Brute Force', severity, ip, 'Repeated failed login attempts', evidence, signa
      ture]);
    }
  }
}
```

Worked example: IP `10.0.0.5` fails at 09:10, 09:11, 09:12, 09:14, 09:19. Starting at 09:10 the window reaches 09:20, so it contains 5 events, which meets the threshold of 5, giving a **Medium** incident. Had there been only 4, nothing fires. The `run(fileId)` function simply calls every rule function in turn (`bruteForce`, `unauthorized`, `abnormalIp`, `sensitivePath`, `errorSpike`, `injection`).

Simple rules (unauthorized, sensitive path, injection) are single queries. Example for injection: fetch entries whose endpoint matches, then test each against `RULES.injection` in JavaScript, since MySQL regex support is limited. Insert one incident per entry with signature `INJECTION|ip|entryId`.

13.7 Dynamic filter builder for incidents

```
exports.list = async (req, res, next) => {
  try {
    const { severity, status, type, ip, from, to } = req.query;
    const page = Math.max(1, parseInt(req.query.page) || 1), limit = 20;
    const where = [], params = [];
    if (severity) { where.push('severity = ?');      params.push(severity); }
    if (status)   { where.push('status = ?');        params.push(status); }
    if (type)     { where.push('incident_type = ?'); params.push(type); }
    if (ip)       { where.push('source_ip = ?');     params.push(ip); }
    if (from)     { where.push('detected_at >= ?');  params.push(from + ' 00:00:00'); }
    if (to)       { where.push('detected_at <= ?');  params.push(to + ' 23:59:59'); }
    const clause = where.length ? 'WHERE ' + where.join(' AND ') : '';
    const [[{ total }]] = await db.query(`SELECT COUNT(*) AS total FROM security_incidents ${clause}`, p
      arams);
    const [rows] = await db.query(
      `SELECT * FROM security_incidents ${clause} ORDER BY detected_at DESC LIMIT ? OFFSET ?`,
      [...params, limit, (page - 1) * limit]);
    res.json({ success:true, data:rows, page, total, message:'Incidents retrieved successfully' });
  } catch (e) { next(e); }
};
```

Column names are fixed in code and only **values** come from the user, which is what keeps this injection-safe. Validate `severity` against `['low','medium','high','critical']` and dates against a date format before use. The report endpoint reuses the same clause with no `LIMIT`.

13.8 Dashboard queries

```
-- cards
SELECT (SELECT COUNT(*) FROM log_files) AS files,
       (SELECT COUNT(*) FROM log_entries) AS entries,
       (SELECT COUNT(*) FROM log_entries WHERE status_code = 401) AS failed,
       (SELECT COUNT(*) FROM security_incidents) AS incidents;
-- top IPs
SELECT source_ip, COUNT(*) AS hits FROM log_entries GROUP BY source_ip ORDER BY hits DESC LIMIT 5;
-- timeline
```

```
SELECT DATE_FORMAT(event_time,'%Y-%m-%d %H:00') AS label, COUNT(*) AS value
FROM log_entries GROUP BY label ORDER BY label;
```

13.9 Error middleware

```
module.exports = (err, req, res, next) => {
  console.error(err);                                    // details stay on the server
  const map = { LIMIT_FILE_SIZE:[400,'File too large'], UNSUPPORTED_TYPE:[400,'Unsupported file type'] }
      ;
  const [status, message] = map[err.code] || map[err.message] || [500, 'Something went wrong'];
  res.status(status).json({ success:false, message, errorCode: err.code || err.message || 'SERVER_ERROR'
       });
};
```

14. Frontend Code Walkthrough

14.1 `login.js`

```
document.getElementById('loginForm').addEventListener('submit', async (e) => {
  e.preventDefault();
  const btn = document.getElementById('loginBtn'), err = document.getElementById('error');
  const username = usernameInput.value.trim(), password = passwordInput.value;
  if (!username || !password) { err.textContent = 'Enter username and password'; return; }
  btn.disabled = true; err.textContent = '';
  try {
    const res = await fetch('/api/auth/login', { method:'POST', headers:{'Content-Type':'application/jso
      n'},
                                                  body: JSON.stringify({ username, password }) });
    const body = await res.json();
    if (!res.ok) throw new Error(body.message);
    sessionStorage.setItem('token', body.data.token);
    location.href = 'dashboard.html';
  } catch (ex) { err.textContent = ex.message; }
  finally { btn.disabled = false; }
});
```

`preventDefault` stops the browser's normal form reload. Disabling the button prevents double submits. `sessionStorage` clears when the tab closes (safer than `localStorage` for a demo).

14.2 `upload.js`

```
const MAX = 10 * 1024 * 1024, OK = ['.log','.txt','.csv'];
async function upload(file) {
  const ext = file.name.slice(file.name.lastIndexOf('.')).toLowerCase();
  if (!file.size) return show('File is empty');
  if (!OK.includes(ext)) return show('Unsupported file type');
  if (file.size > MAX) return show('File exceeds 10 MB');
  const fd = new FormData(); fd.append('logFile', file);
  const res = await api('/logs/upload', { method:'POST', body: fd });
  show(`File ID ${res.data.fileId}: ${res.data.status}. Valid: ${res.data.validLines}, invalid: ${res.da
      ta.invalidLines}`);
  loadHistory();
}
```

`fetch` cannot show upload progress; use `XMLHttpRequest` with `xhr.upload.onprogress` if you want a real progress bar.

14.3 `incidents.js`

```
async function loadIncidents(page = 1) {
  const q = new URLSearchParams({ page });
  ['severity','status','type','ip','from','to'].forEach(k => { const v = document.getElementById(k).valu
      e; if (v) q.set(k, v); });
  const res = await api('/incidents?' + q);
  const tbody = document.getElementById('rows'); tbody.innerHTML = '';
  if (!res.data.length) { tbody.innerHTML = '<tr><td colspan="8">No incidents found</td></tr>'; return;
      }
  res.data.forEach(i => {
    const tr = document.createElement('tr');
    [i.id, i.detected_at, i.incident_type, i.source_ip, i.severity, i.description, i.status]
      .forEach(v => { const td = document.createElement('td'); td.textContent = v; tr.appendChild(td); }
      );  // safe from XSS
    tbody.appendChild(tr);
  });
  renderPagination(res.page, Math.ceil(res.total / 20));
```

```
}
```

`URLSearchParams` builds and encodes the query string; only non-empty filters are included.

14.4 `dashboard.js`

```
const [summary, timeline, severity] = await Promise.all([api('/dashboard/summary'), api('/dashboard/time
      line'), api('/dashboard/severity')]);
document.getElementById('cardIncidents').textContent = summary.data.incidents;
new Chart(document.getElementById('sevChart'), { type:'doughnut', data:{ labels:severity.data.labels, da
      tasets:[{ data:severity.data.values }] } });
new Chart(document.getElementById('timeChart'), { type:'line', data:{ labels:timeline.data.labels, datas
      ets:[{ label:'Events', data:timeline.data.values }] } });
```

`Promise.all` fires the three requests in parallel, so the page loads faster than three sequential calls.

15. Sample Test Data with Expected Results

failed_login.log (expected: 6 entries, 1 incident: Brute Force, Medium)

```
10.0.0.5 - - [01/Sep/2026:09:10:01 +0530] "POST /login HTTP/1.1" 401 120
10.0.0.5 - - [01/Sep/2026:09:11:12 +0530] "POST /login HTTP/1.1" 401 120
10.0.0.5 - - [01/Sep/2026:09:12:30 +0530] "POST /login HTTP/1.1" 401 120
10.0.0.5 - - [01/Sep/2026:09:14:05 +0530] "POST /login HTTP/1.1" 401 120
10.0.0.5 - - [01/Sep/2026:09:19:40 +0530] "POST /login HTTP/1.1" 401 120
10.0.0.9 - - [01/Sep/2026:09:15:00 +0530] "GET /index.html HTTP/1.1" 200 2048
```

normal_access.log (expected: all entries parsed, 0 incidents): only `200`/`304` responses on ordinary pages.

invalid_format.log (expected: 0 valid, status `failed`): lines such as `hello world`.

Note that the login attempts above also trigger the Unauthorized Access rule (401), so record that expected count too.

16. Design Decisions and Their Reasons

Decision	Reason
Rules in a config object	Thresholds can be tuned and explained in one place
Signature + unique key	The same attack is one incident, not thousands
Store evidence text	Every alert can be justified at evaluation
SQL aggregation	Databases are far faster at counting than browser JavaScript
Streaming parse + batch insert	Constant memory, far fewer DB round trips
Services separate from routes	Detection logic can be unit-tested without a server
Wording "suspicious", not "attack"	Pattern matching cannot prove intent, so the tool reports honestly

17. Glossary

- **JWT**: a signed token proving who the user is.
- **Middleware**: a function that runs between the request and the final handler.
- **Parameterized query**: SQL with `?` placeholders so user input is never treated as code.
- **Sliding window**: counting events inside a moving time range.
- **False positive**: a normal event wrongly flagged.
- **XSS**: injecting script into a page through unescaped content.
- **Path traversal**: using `../` in names to reach files outside the intended folder.